

Report Lab08:Table Relationships

พิธีษฐ์ ทรัพย์อุดมโชติ 6733803285-2 Sec.1

ส่วนที่ 1: หลักการออกแบบ

SOLID Principles ที่ใช้ในโปรเจกต์

S — Single Responsibility Principle

แต่ละคลาสมีหน้าที่เดียว

1. Product เก็บข้อมูลสินค้าหลัก
2. ProductDetail เก็บเฉพาะข้อมูลเสริม (description, warranty, weight, dimensions)
3. Review เก็บเฉพาะข้อมูลรีวิว — ไม่ยึดทุกอย่างไว้ใน Entity เดียว
4. ProductService รับผิดชอบแค่ business logic
5. ProductController รับผิดชอบแค่แปลง HTTP request/response

O — Open/Closed Principle

DiscountContext ออกแบบให้ "เปิดรับการขยาย ปิดรับการแก้ไข" — ถ้าจะเพิ่มโปรโมชันใหม่ เช่น ClearanceStrategy แค่สร้างคลาสใหม่ที่ implement DiscountStrategy แล้วติด @Component เท่านั้น ไม่ต้องแก้โค้ดใน DiscountContext, ProductService, หรือ Product เลยแม้แต่บรรทัดเดียว เพราะ Spring จะ inject strategy ใหม่เข้า list อัตโนมัติ เช่นเดียวกับการเพิ่ม Entity Review เข้ามาใน Lab นี้ ก็ไม่ต้องแก้ Product/ProductDetail เดิมที่มีจาก Lab 7

L — Liskov Substitution Principle

DiscountStrategy ทั้ง 3 ตัว (NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy) ใช้แทนกันได้หมดผ่าน interface เดียวกัน โดยที่ DiscountContext ไม่ต้องรู้เลยว่ากำลังเรียก strategy ตัวไหนอยู่ ผลลัพธ์ที่ได้ยังคงถูกต้องตามสัญญาของ interface เสมอ (สินค้า double ที่เป็นราคาหลังหักส่วนลด)

I — Interface Segregation Principle

แยก ProductRepository, ProductDetailRepository, ReviewRepository ออกเป็นคนละ interface แทนที่จะรวมทุก query ไว้ที่เดียว — แต่ละ interface มีเฉพาะ method ที่ตัวเองต้องใช้จริงๆ เช่น ReviewRepository มี findByProductId เพิ่มมาเฉพาะของตัวเอง โดยไม่ไปทำให้ ProductRepository ต้องมี method ที่ไม่เกี่ยวข้อง

D — Dependency Inversion Principle

ProductService ไม่ได้ new ProductRepository() หรือ new DiscountContext() เอง แต่รับเข้ามาผ่าน **Constructor Injection**:

Java

```
public ProductService(ProductRepository productRepository, DiscountContext discountContext) {  
    this.productRepository = productRepository;  
}
```

```
this.discountContext = discountContext;  
}
```

Service จึงขึ้นอยู่กับ abstraction (interface ProductRepository, DiscountStrategy) ไม่ใช่ implementation ที่เจาะจง — ทำให้ทดสอบและสลับ implementation ได้ง่ายโดยไม่กระทบโค้ดชั้นบน เช่นเดียวกับ ProductController ที่รับ ProductService ผ่าน constructor เช่นกัน

ความแตกต่างระหว่าง 1:1 กับ 1:N

หัวข้อ	1:1 (Product ↔ ProductDetail)	1:N (Product → Review)
ความหมาย	สินค้า 1 ชิ้น มีข้อมูลเสริมได้แค่ 1 ชุด	สินค้า 1 ชิ้น มีรีวิวได้หลายรายการไม่จำกัด
FK อยู่ที่ไหน	ฝั่งเจ้าของ (products.detail_id)	ฝั่ง Many เสมอ (reviews.product_id)
Annotation	@OneToOne + @JoinColumn (owning side), @OneToOne(mappedBy=...) (inverse side)	@ManyToOne + @JoinColumn (owning side, ฝั่ง Review), @OneToMany(mappedBy=...) (inverse side, ฝั่ง Product)
ควรใช้เมื่อไหร่	ข้อมูลเสริมที่เป็น "ส่วนขยาย" ของ entity หลัก แยกออกมาเพื่อความเป็นระเบียบ (SRP) เช่น สเปกสินค้า ไม่ได้ใช้ทุก query — จะได้ไม่ต้อง join ทุกครั้งถ้าไม่จำเป็น	ข้อมูลแบบ child ที่เพิ่มได้เรื่อยๆ ไม่จำกัดจำนวน และแต่ละรายการเป็นอิสระต่อกัน (รีวิวแต่ละอันไม่ผูกกัน)

สรุปสั้นๆ: 1:1 ใช้เมื่อ "แตกข้อมูลออกจากกันแต่ยังคงเป็น 1 คู่กันเสมอ" ส่วน 1:N ใช้เมื่อฝั่งหนึ่งมีจำนวนได้มากกว่า 1 แบบไม่จำกัด — กฎที่จำง่ายคือ FK อยู่ฝั่ง "Many" เสมอ ไม่ว่าจะเป็น 1:N หรือ M:N

Strategy Pattern สำหรับคำนวณส่วนลด

ปัญหาที่ Strategy Pattern แก้: ถ้าเขียน if/else เช็ค discountType ไว้ในทีเดียว (เช่นใน service) ทุกครั้งที่มีโปรโมชั่นใหม่ต้องแก้ไขโค้ดเดิม เสี่ยงกระทบของเก่า (ขัดกับ OCP)

โครงสร้างที่ใช้:

DiscountStrategy (interface) — สัญญากลางว่าทุก strategy ต้องมี applyDiscount(price) และ getName()

Concrete strategies — NoDiscountStrategy (0%), MemberDiscountStrategy (10%), SeasonalSaleStrategy (20%) แต่ละตัวติด @Component ให้ Spring จัดการเป็น bean

DiscountContext — ตัวกลาง (context) ที่รับ List<DiscountStrategy> ทุกตัวเข้ามาผ่าน constructor (Spring inject ให้อัตโนมัติ) แล้วเก็บลง Map<String, DiscountStrategy> โดยใช้ getName() เป็น key จากนั้นเลือก strategy ที่ตรงกับ discountType ของสินค้ามาคำนวณ:

Java

```
public double calculatePrice(double price, String discountType) {
    DiscountStrategy strategy = strategies.getOrDefault(discountType, strategies.get("NONE"));
    return strategy.applyDiscount(price);
}
```

ข้อดี: ถ้าจะเพิ่มโปรโมชั่นใหม่ แค่เพิ่มคลาสใหม่ implement DiscountStrategy โดยไม่ต้องแตะ DiscountContext เลย — ตรงตามหลัก OCP และ DIP ไปพร้อมกัน

Execution Flow ตั้งแต่ HTTP Request → DB

ตัวอย่าง: ผู้ใช้กด "บันทึกสินค้า" ในหน้า add.html (POST /products/save)

1. **Browser** ส่ง HTTP POST request พร้อม form data (name, category, detail.description, reviews[0].reviewer ฯลฯ) ไปที่ /products/save
2. **DispatcherServlet** (ของ Spring MVC) รับ request แล้วหา handler ที่ map กับ URL นี้
3. **ProductController.save()** ถูกเรียก — Spring @ModelAttribute จะสร้าง object Product ใหม่ แล้ว bind ค่าจาก form เข้าไปอัตโนมัติ (รวมถึงสร้าง ProductDetail และ Review ซ้อนข้างในให้ผ่าน nested property binding)
4. Controller เรียก **productService.save(product)** — ส่งต่อไปยัง business logic layer โดยไม่รู้รายละเอียดการ persist
5. **ProductService.save()** ทำความสะอาดข้อมูล (ลบรีวิวที่กรอกว่างทิ้ง, เชื่อม back-reference ระหว่าง Product ↔ Review, ใส่วันที่รีวิวถ้าไม่ได้กรอก) แล้วเรียก **productRepository.save(product)**
6. **ProductRepository** (Spring Data JPA proxy) แปลง object เป็น SQL ผ่าน **Hibernate** — เพราะตั้ง cascade = CascadeType.ALL ไว้ที่ detail และ reviews การ save ครั้งเดียวจึง insert ลงทั้ง 3 ตาราง (products, product_details, reviews) พร้อมกันในทรานแซกชันเดียว
7. **PostgreSQL** รับคำสั่ง INSERT จริง บันทึกลง disk แล้วส่ง generated ID กลับ
8. Controller ได้ Product ที่มี ID แล้ว ส่ง **redirect** กลับไป /products พร้อม flash message
9. Browser ยิง GET /products รอบใหม่ → ProductController.list() → productService.findAll() → query ทั้งหมดกลับมาแสดงในตาราง (list.html)

ส่วนที่ 2: Code + คำอธิบาย

Entity 1: Product.java

```
Java

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType = "NONE";

    // — 1:1 กับ ProductDetail — Product เป็นฝั่งเจ้าของ (มี FK detail_id) —
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    // — 1:N กับ Review — FK อยู่ฝั่ง Review เสมอ —
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.EAGER)
    private List<Review> reviews = new ArrayList<>();

    // ... constructor, getter/setter
}
```

annotation:

- `@Entity` — บอก JPA ว่าคลาสนี้แม็ปกับตารางในฐานข้อมูล
- `@Table(name = "products")` — กำหนดชื่อตารางจริงใน PostgreSQL
- `@Id` + `@GeneratedValue(strategy = GenerationType.IDENTITY)` — กำหนด primary key และให้ database เป็นผู้ auto-increment ค่าให้ (ใช้ SERIAL/IDENTITY ของ PostgreSQL)
- `@Column(nullable = false)` — บังคับว่า name ห้ามเป็นค่าว่างในระดับ database schema
- `@OneToOne(cascade = CascadeType.ALL)` — ประกาศความสัมพันธ์ 1:1 ฝั่งเจ้าของ cascade = ALL หมายความว่าถ้า save/delete Product จะ save/delete ProductDetail ตามไปด้วยอัตโนมัติ ไม่ต้องเรียก repository ของ ProductDetail แยก
- `@JoinColumn(name = "detail_id")` — บอกว่า FK column ชื่อ detail_id ถูกเก็บไว้ที่ตาราง products เอง (Product เป็น owning side)

- `@OneToMany(mappedBy = "product", orphanRemoval = true)` — ประกาศความสัมพันธ์ 1:N ฝั่ง `inverse` (Product ไม่ถือ FK) `mappedBy = "product"` ซึ่งไปที่ field ชื่อ `product` ใน class `Review` ว่าเป็นเจ้าของความสัมพันธ์จริง ส่วน `orphanRemoval = true` ทำให้ถ้าลบ `Review` ออกจาก list แล้ว `save` จะลบแถวนั้นออกจาก database ด้วย ไม่ใช่แค่ตัดความสัมพันธ์

Entity 2: ProductDetail.java

```
Java
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 1000)
    private String description;

    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    // inverse side ของ 1:1 — ไม่ถือ FK จริง แค่ชี้กลับไปหา Product
    @OneToOne(mappedBy = "detail")
    private Product product;
    // ... constructor, getter/setter
}
```

annotation:

- `@Column(length = 1000)` — กำหนดความยาวสูงสุดของ column `description` ในฐานข้อมูลเป็น `VARCHAR(1000)` แทนค่า default (255) เพราะรายละเอียดสินค้าอาจยาว
- `@OneToOne(mappedBy = "detail")` — ฝั่ง **inverse side** ของความสัมพันธ์ 1:1 คำว่า `mappedBy = "detail"` บอกว่า field ชื่อ `detail` ใน class `Product` ต่างหากที่เป็นเจ้าของความสัมพันธ์ (ถือ FK จริง) ฝั่งนี้จึงไม่มี `@JoinColumn` เพราะไม่ได้ถือ FK — มีไว้แค่ให้เขียนโค้ดแบบ bi-directional (จาก `ProductDetail` เรียกกลับไปหา `Product` ได้)

Entity 3: Review.java

```
Java
@Entity
@Table(name = "reviews")
public class Review {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String reviewer;
private Integer rating;

@Column(length = 1000)
private String comment;

private LocalDate reviewDate;

@ManyToOne
@JoinColumn(name = "product_id")
@JsonIgnore
private Product product;
// ... constructor, getter/setter
}

```

annotation:

- @ManyToOne — ประกาศว่า Review หลายแถวสามารถชี้ไปหา Product เดียวกันได้ (ฝั่ง "Many" ของความสัมพันธ์ 1:N) ฝั่งนี้เป็น **owning side** เสมอ ตามกฎ JPA ว่า FK ต้องอยู่ฝั่ง Many
- @JoinColumn(name = "product_id") — สร้าง FK column ชื่อ product_id ในตาราง reviews ชี้ไปที่ products.id
- @JsonIgnore — ป้องกัน infinite loop ถ้ามีการแปลง object เป็น JSON ในอนาคต (Product → reviews → product → reviews → ... จนไม่รู้จบ) เป็น best practice ทั่วไปเวลาที่มี bi-directional relationship

Service Layer — ProductService.java

```

Java

@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    // Constructor Injection — Spring จะ inject bean ทั้งสองให้อัตโนมัติ
    public ProductService(ProductRepository productRepository,
                          DiscountContext discountContext) {
        this.productRepository = productRepository;
        this.discountContext = discountContext;
    }
}

```

```

public Product save(Product product) {
    cleanBlankReviews(product);
    linkReviews(product);
    return productRepository.save(product);
}
// ... findAll, findById, update, deleteById, calculateFinalPrice
}

```

Constructor Injection:

- @Service บอก Spring ว่าคลาสนี้คือ business logic layer ให้สร้างเป็น bean ไว้ในตอน context startup
- แทนที่จะเขียน new ProductRepository() เองในโค้ด (ซึ่งจะผูกติดกับ implementation ที่เจาะจงและทดสอบยาก) เราประกาศ dependency (ProductRepository, DiscountContext) เป็น private final field แล้วรับเข้ามาทาง **constructor** — Spring container จะมองเห็น constructor นี้ตอนสร้าง bean ProductService แล้วหา bean ที่ type ตรงกันมา inject ให้อัตโนมัติ (ไม่ต้องเขียน @Autowired เพราะ Spring 4.3+ ขึ้นไป ถ้าคลาสมี constructor เดียวจะ inject ให้เองโดยไม่ต้องระบุ)
- ข้อดีเทียบกับ Field Injection (@Autowired private ProductRepository productRepository;):
 1. ทำให้ field เป็น final ได้ → immutable, ป้องกันโค้ดที่อื่นมาสลับ dependency กลางคัน
 2. ถ้าลืม inject dependency ที่จำเป็น จะ compile error ทันที (ไม่ใช่ runtime NullPointerException)
 3. เขียน unit test ได้ง่าย เพราะสร้าง new ProductService(mockRepo, mockContext) ส่ง mock object เข้าไปตรงๆ ได้เลยโดยไม่ต้องพึ่ง Spring container

Controller Layer — ProductController.java

```
Java

@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @PostMapping("/save")
    public String save(@ModelAttribute("product") Product product,
        RedirectAttributes redirectAttributes) {
        productService.save(product);
        redirectAttributes.addFlashAttribute("message",
            "เพิ่มสินค้า \"\" + product.getName() + "\"" + " สำเร็จ");
        return "redirect:/products";
    }

    // ... list, showAddForm, showEditForm, update, showDeleteConfirm, delete
}
```

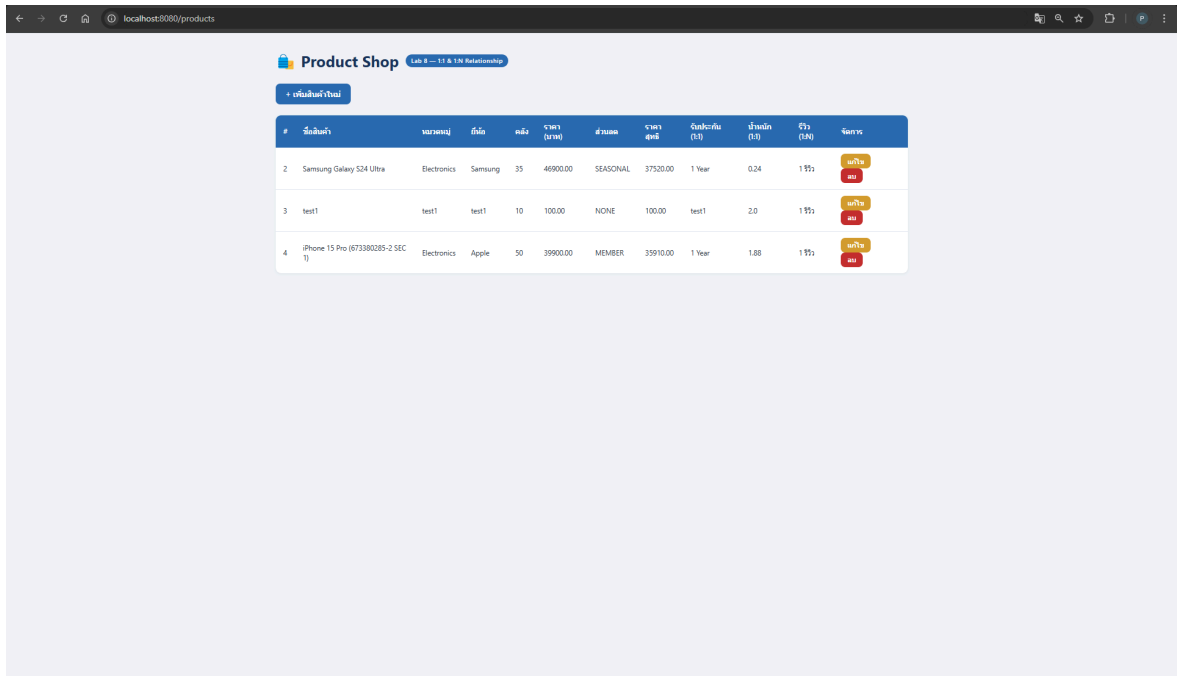
Constructor Injection:

- @Controller + @RequestMapping("/products") บอกว่าคลาสนี้รับผิดชอบทุก URL ที่ขึ้นต้นด้วย /products และ method ที่ return เป็น String จะถูกตีความเป็นชื่อ view (Thymeleaf template) ไม่ใช่ raw response

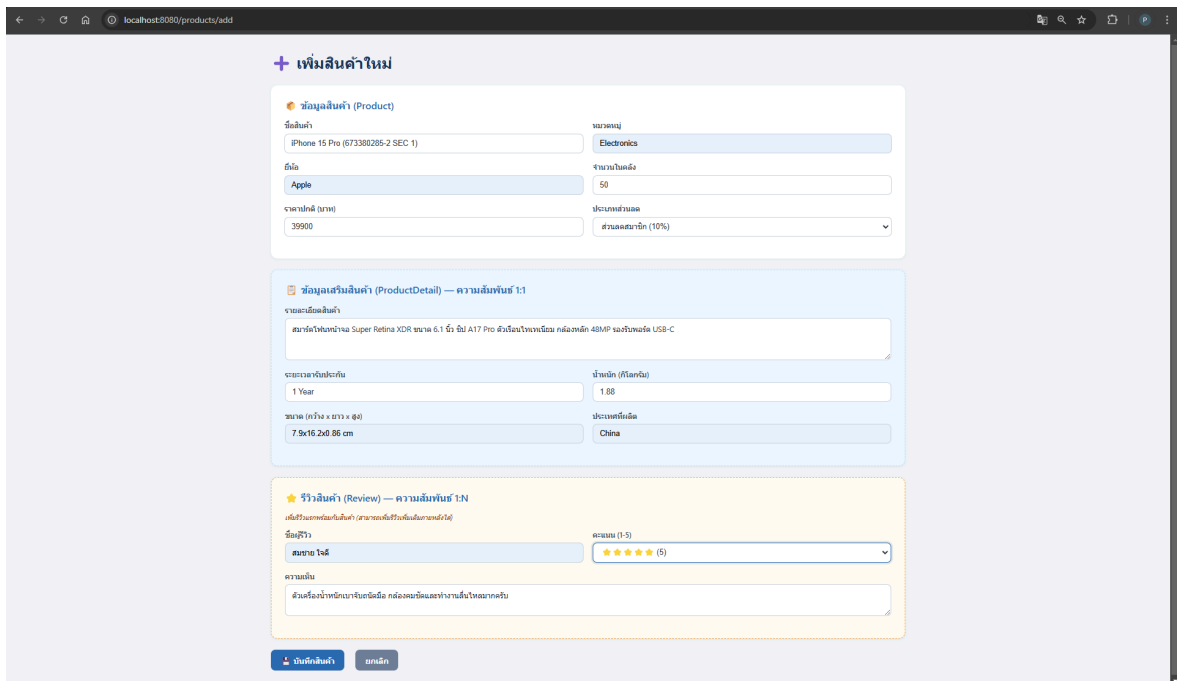
เช่นเดียวกับ ProductService, Controller รับ ProductService ผ่าน constructor แทนการสร้างเอง ทำให้ Controller ขึ้นกับ **abstraction** ของ **business logic** เท่านั้น ไม่รู้เลยว่าเบื้องหลัง service ไปคุยกับ repository/database ยังไง — สอดคล้องกับหลัก DIP และทำให้ layer ต่างๆ (Controller → Service → Repository) แยกกันอย่างชัดเจน แก้ไข/ทดสอบแต่ละชั้นแยกจากกันได้โดยไม่กระทบกัน

ส่วนที่ 3: ภาพหน้าจอ

product



create



Edit

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380285-2 SEC 1)

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนในสต็อก

50

ราคาปกติ (บาท)

39900.0

ส่วนลดสมาชิก (10%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

สมาร์ตโฟนรุ่น Super Retina XDR ขนาด 6.1 นิ้ว chip A17 Pro ตัวเป็นไทเทเนียม กล้องหลัก 48MP รองรับพอร์จ USB-C

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

1.88

ขนาด

7.9x16.2x0.86 cm

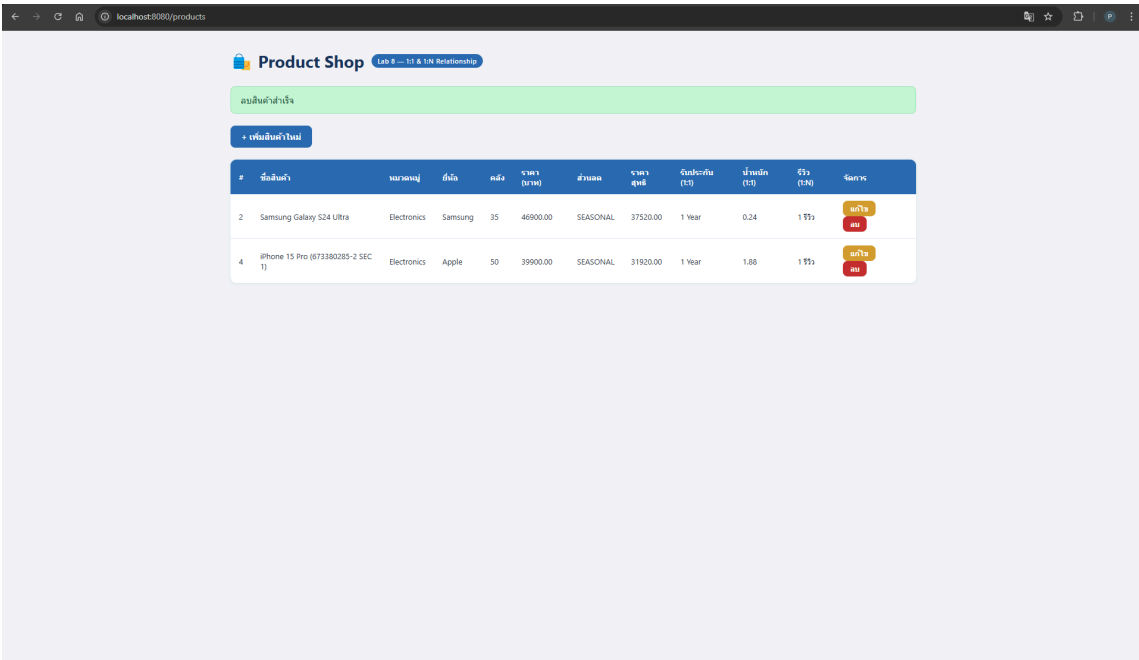
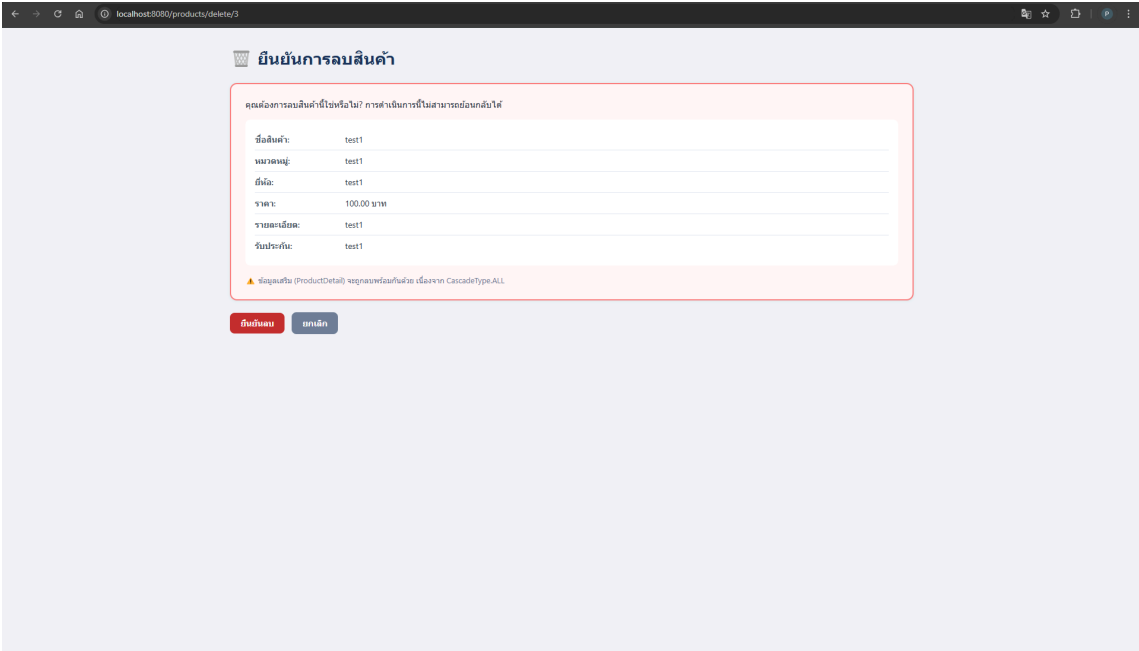
ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก

Delete



Product

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'public.products' table selected under the 'public' schema. The main query editor contains the following SQL query:

```
1 SELECT * FROM public.products
2 ORDER BY id ASC
```

The Data Output pane at the bottom displays the results of the query in a table format. The table has 8 columns: id, brand, category, discount_type, name, price, stock, and detail_id. The results show two rows of product data.

id	brand	category	discount_type	name	price	stock	detail_id
1	2	Samsung	Electronics	SEASONAL	Samsung Galaxy S24 Ultra	46900	35
2	4	Apple	Electronics	SEASONAL	iPhone 15 Pro (6/3380285 2 SEC 1)	39900	50

ProductDetail

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'public.product_details' table selected under the 'public' schema. The main query editor contains the following SQL query:

```
1 SELECT * FROM public.product_details
2 ORDER BY id ASC
```

The Data Output pane at the bottom displays the results of the query in a table format. The table has 7 columns: id, description, dimensions, manufacture_country, warranty, and weight. The results show two rows of product detail data.

id	description	dimensions	manufacture_country	warranty	weight
1	2	หน้าจอ AMOLED 2X ขนาด 6.8 นิ้ว Refresh Rate 120Hz	Vietnam	1 Year	0.24
2	4	กล้องหลัก 48MP Super Retina XDR ขนาด 6.1 นิ้ว A17 Pro	China	1 Year	1.88

Review

